

```
In [1]: pip install nltk
```

```
Requirement already satisfied: nltk in c:\users\dell\anaconda3\lib\site-packages (3.9.2)  
Requirement already satisfied: click in c:\users\dell\anaconda3\lib\site-packages (from nltk) (8.2.1)  
Requirement already satisfied: joblib in c:\users\dell\anaconda3\lib\site-packages (from nltk) (1.5.2)  
Requirement already satisfied: regex>=2021.8.3 in c:\users\dell\anaconda3\lib\site-packages (from nltk) (2025.9.1)  
Requirement already satisfied: tqdm in c:\users\dell\anaconda3\lib\site-packages (from nltk) (4.67.1)  
Requirement already satisfied: colorama in c:\users\dell\anaconda3\lib\site-packages (from click->nltk) (0.4.6)  
Note: you may need to restart the kernel to use updated packages.
```

```
In [2]: import nltk
```

```
In [3]: from nltk.tokenize import sent_tokenize
```

```
In [4]: from nltk.tokenize import word_tokenize
```

```
In [5]: from nltk.corpus import stopwords  
from nltk.stem import PorterStemmer  
from nltk.stem import WordNetLemmatizer  
import re  
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
In [6]: nltk.download('punkt')  
nltk.download('stopwords')  
nltk.download('wordnet')  
nltk.download('averaged_perceptron_tagger')
```

```
[nltk_data] Downloading package punkt to  
[nltk_data] C:\Users\Dell\AppData\Roaming\nltk_data...  
[nltk_data] Unzipping tokenizers\punkt.zip.  
[nltk_data] Downloading package stopwords to  
[nltk_data] C:\Users\Dell\AppData\Roaming\nltk_data...  
[nltk_data] Unzipping corpora\stopwords.zip.  
[nltk_data] Downloading package wordnet to  
[nltk_data] C:\Users\Dell\AppData\Roaming\nltk_data...  
[nltk_data] Downloading package averaged_perceptron_tagger to  
[nltk_data] C:\Users\Dell\AppData\Roaming\nltk_data...  
[nltk_data] Unzipping taggers\averaged_perceptron_tagger.zip.
```

```
Out[6]: True
```

```
In [7]: text="Tokenization replaces a sensitive data element, for example, a bank account"
```

```
In [10]: nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to  
[nltk_data] C:\Users\Dell\AppData\Roaming\nltk_data...  
[nltk_data] Unzipping tokenizers\punkt_tab.zip.
```

```
Out[10]: True
```

```
In [11]: sentences=sent_tokenize(text)
print(sentences)
```

['Tokenization replaces a sensitive data element, for example, a bank account number, with a non-sensitive substitute, known as a token.', 'The token is a randomized data string that has no essential or exploitable value or meaning.', 'It is a unique identifier which retains all the pertinent information about the data without compromising its security']

```
In [13]: words=word_tokenize(text)
print(words)
```

['Tokenization', 'replaces', 'a', 'sensitive', 'data', 'element', ',', 'for', 'example', ',', 'a', 'bank', 'account', 'number', ',', 'with', 'a', 'non-sensitive', 'substitute', ',', 'known', 'as', 'a', 'token', '.', 'The', 'token', 'is', 'a', 'randomized', 'data', 'string', 'that', 'has', 'no', 'essential', 'or', 'exploitable', 'value', 'or', 'meaning', '.', 'It', 'is', 'a', 'unique', 'identifier', 'which', 'retains', 'all', 'the', 'pertinent', 'information', 'about', 'the', 'data', 'without', 'compromising', 'its', 'security']

```
In [15]: stop_words=set(stopwords.words('english'))
print(stop_words)
```

{'under', 'and', 'hadn', "needn't", 'further', 'not', 'doesn', 'but', 'this', 'against', 'yourselves', 'being', 'what', "wasn't", 'isn', 'ain', 'which', 'was', "mightn't", 'nor', 'that', 'the', 'now', 'between', "it'll", 'as', 'there', 'theirs', 'mustn', "you'll", 'their', "weren't", 'in', "doesn't", 'some', "you're", 'won', 'll', 'shan', 'have', 'they', 'when', 'weren', "hadn't", 'whom', 'before', "we'll", 'through', "couldn't", 'such', 'below', 'i've', 'myself', 'because', 'had', "they're", 'yourself', 'no', 'of', 'you', 'its', "they'd", 'most', "should've", 'couldn', 's', "aren't", "we're", 'for', 've', "wouldn't", 'he'd', 'we', 'or', 'mightn', 'o', 'own', 'she', 'very', 'i', 'both', 'if', 'into', 'about', 'an', 'm', "hasn't", 'same', "that'll", 'did', 'few', 'yours', "you'd", 'from', "they'll", 'how', 'while', "didn't", 'her', 'itself', 'just', 'should', 'too', "she's", 'are', "he's", 'why', 'ourselves', 'my', 'on', "shouldn't", 'than', "i'll", 'does', 'other', 'those', 'down', 'ma', 'them', 'herself', 'hasn', "she'd", 'it's', 'can', 'doing', "haven't", 'at', "won't", 'he', "it'd", 'only', 'to', "she'll", 'be', 'over', "we'd", "don't", 'once', 'out', 'wasn', 'didn', 'it', 'himself', "they've", 'here', 'has', 'so', 'his', 're', 'having', 'wouldn', 'is', 'themselves', 'your', "i'd", "isn't", 'where', 'any', 'during', 'him', 'am', 'aren', 'our', "we've", 'up', "you've", 'above', 'after', 'shouldn', 'then', 'will', 'again', "he'll", 'by', 'ours', 'a', 'd', 'these', 'until', 't', 'y', 'each', 'don', "mustn't", 'were', 'more', 'who', 'haven', 'me', 'needn', "i'm", 'been', 'off', "shan't", 'all', 'with', 'do', 'hers'}

```
In [16]: text="How to remove stopwords with NLTK library in PYTHON?"
text=re.sub('[^a-zA-Z]', ' ',text)
tokens=word_tokenize(text.lower())
filtered_text=[]
for w in tokens:
    if w not in stop_words:
        filtered_text.append(w)
print('tokenized text:',tokens)
print('filtered text:',filtered_text)
```

tokenized text: ['how', 'to', 'remove', 'stopwords', 'with', 'nltk', 'library', 'in', 'python']

filtered text: ['remove', 'stopwords', 'nltk', 'library', 'python']

```
In [21]: nltk.download('averaged_perceptron_tagger_eng')
words1=['ram','before','eating','waiting']
```

```
for w in words1:
    print(w,":",nlk.pos_tag([w]))
```

```
[nlk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] C:\Users\Dell\AppData\Roaming\nltk_data...
[nltk_data] Unzipping taggers\averaged_perceptron_tagger_eng.zip.
ram : [('ram', 'NN')]
before : [('before', 'IN')]
eating : [('eating', 'VBG')]
waiting : [('waiting', 'VBG')]
```

```
In [23]: ps=PorterStemmer()
for w in words1:
    print(w,':',ps.stem(w))
```

```
ram : ram
before : befor
eating : eat
waiting : wait
```

```
In [25]: lemmatizer=WordNetLemmatizer()
for w in words1:
    print(w,':',lemmatizer.lemmatize(w,pos='v'))
```

```
ram : ram
before : before
eating : eat
waiting : wait
```

```
In [26]: paragraph= """India is a vast country with second highest populati
on in the world. It is a country
with diverse cultures, traditions and beliefs. People in India cel
ebrate unity in diversity.
Festivals like Diwali, Holi, Navratri, Ramzan, Christmas etc. are
celebrated by people across India
and create a sense of brotherhood and cultural unity. Each festiva
l has its religious and cultural importance."""
```

```
In [31]: wn=WordNetLemmatizer()
sentences=sent_tokenize(paragraph)
cleaned_para=[]
for i in range (len(sentences)):
    review=re.sub('[^a-zA-Z]', ' ',sentences[i])
    review=review.lower()
    review=review.split()
    review = [wn.lemmatize(word) for word in review if not word in set(stopwords
    review=' '.join(review)
    cleaned_para.append(review)
print(cleaned_para)
```

```
['india vast country second highest populati world', 'country diverse culture tra
dition belief', 'people india cel ebrate unity diversity', 'festival like diwali
holi navratri ramzan christmas etc', 'celebrated people across india create sense
brotherhood cultural unity', 'festiva l religious cultural importance']
```

```
In [33]: tfidf=TfidfVectorizer()
x=tfidf.fit_transform(cleaned_para).toarray()
print(x)
```

```

[[0.      0.      0.      0.      0.      0.
  0.33061545 0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.      0.40318254
  0.      0.      0.27912828 0.      0.      0.
  0.40318254 0.      0.      0.40318254 0.      0.
  0.      0.40318254 0.40318254]
[0.      0.46262479 0.      0.      0.      0.
  0.37935895 0.      0.      0.46262479 0.46262479 0.
  0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.      0.46262479
  0.      0.      0.      ]
[0.      0.      0.      0.45529187 0.      0.
  0.      0.      0.      0.      0.      0.45529187
  0.      0.45529187 0.      0.      0.      0.
  0.      0.      0.31520422 0.      0.      0.37334585
  0.      0.      0.      0.      0.      0.
  0.37334585 0.      0.      ]
[0.      0.      0.      0.      0.      0.35355339
  0.      0.      0.      0.      0.      0.
  0.35355339 0.      0.35355339 0.      0.35355339 0.
  0.35355339 0.      0.      0.35355339 0.35355339 0.
  0.      0.35355339 0.      0.      0.      0.
  0.      0.      0.      ]
[0.36523197 0.      0.36523197 0.      0.36523197 0.
  0.      0.36523197 0.29949544 0.      0.      0.
  0.      0.      0.      0.      0.      0.
  0.      0.      0.25285463 0.      0.      0.29949544
  0.      0.      0.      0.      0.36523197 0.
  0.29949544 0.      0.      ]
[0.      0.      0.      0.      0.      0.
  0.      0.      0.42790272 0.      0.      0.
  0.      0.      0.      0.52182349 0.      0.
  0.      0.52182349 0.      0.      0.      0.
  0.      0.      0.52182349 0.      0.      0.
  0.      0.      0.      ]]

```

In []: